

Arama Algoritmaları

Hash Tablosu (Hash Table) ve Hash Fonksiyonlarıyla Arama

Hash tablosu, bir anahtar (key) ve değer (value) çiftlerini saklamak için kullanılan bir veri yapısıdır. Anahtarlar benzersizdir ve her anahtarın bir değeri vardır. Hash tablosu, hızlı arama, ekleme ve silme işlemleri sağlamak için kullanılır. Anahtarlar, bir hash fonksiyonu kullanılarak bir hash değerine dönüştürülür ve bu hash değeri bir dizi indeksi olarak kullanılarak anahtar-değer çiftleri bir diziye (array) yerleştirilir.

Hash tablosu kullanırken önemli olan bazı kavramlar şunlardır:

1. Hash Fonksiyonu (Hash Function): Anahtarları hash değerlerine dönüştüren bir fonksiyondur. İdeal bir hash fonksiyonu, farklı anahtarlar için farklı hash değerleri üretir ve hash değerlerinin dağılımı homojen olur.
2. Collision (Çakışma): İdeal olmayan durumlarda, farklı anahtarlar aynı hash değerine sahip olabilir. Bu duruma "çakışma" denir. Çakışmaları çözmek için farklı yöntemler kullanılabilir, örneğin zincirleme (chaining) veya açık adresleme (open addressing).
3. Zincirleme (Chaining): Çakışan anahtar-değer çiftleri aynı hash değerine sahip olan bir dizi indeksine yerleştirilir. Her indeks bir bağlı listeyi işaret eder ve çakışan çiftler bu bağlı listelere eklenir.
4. Açık Adresleme (Open Addressing): Çakışma durumunda, başka bir boş indeks bulana kadar alternatif bir indekse gidilir. Farklı açık adresleme yöntemleri vardır, örneğin lineer arama, karesel arama, çiftleme gibi.

Hash tablosu, hızlı arama işlemleri sağlar ve bu nedenle büyük veri kümeleri üzerinde etkilidir. Ancak, uygun bir hash fonksiyonu seçimi ve çakışma yönetimi önemlidir.

Python Kodu: Şimdi, bu açıklamalar ışığında, Python'da Hash Tablosu kullanarak arama işlemini gerçekleştiren bir örnek kod görelim:


```

In [2]: class HashTable:
    def __init__(self, size):
        """
        Hash tablosu sınıfının yapıcı metodudur.

        Parametreler:
        size (int): Hash tablosunun boyutu.
        """
        self.size = size
        self.hash_table = [None] * size # Belirlenen boyutta bir hash t

    def hash_function(self, key):
        """
        Verilen anahtarın hash değerini hesaplar.

        Parametreler:
        key: Hesaplanacak anahtar.

        Dönüş:
        int: Anahtarın hash değeri.
        """
        return key % self.size # Anahtarın boyuta göre hash değeri hesa

    def put(self, key, value):
        """
        Hash tablosuna anahtar-değer çifti ekler.

        Parametreler:
        key: Eklenecek anahtar.
        value: Eklenecek değer.
        """
        index = self.hash_function(key) # Anahtarın hash değeri alınır
        if self.hash_table[index] is None: # Eğer ilgili indekste bir v
            self.hash_table[index] = [(key, value)] # Yeni bir liste ol
        else: # Eğer ilgili indekste bir veri varsa
            for i in range(len(self.hash_table[index])):
                if self.hash_table[index][i][0] == key: # Anahtar zaten
                    self.hash_table[index][i] = (key, value) # Değer gü
                    return
            self.hash_table[index].append((key, value)) # Yeni anahtar-

    def get(self, key):
        """
        Verilen anahtara karşılık gelen değeri döndürür.

        Parametreler:
        key: Aranacak anahtar.

        Dönüş:
        Değer: Aranacak anahtara karşılık gelen değer, eğer anahtar bulunamazsa
        """
        index = self.hash_function(key) # Anahtarın hash değeri alınır
        if self.hash_table[index] is not None: # Eğer ilgili indekste b
            for item in self.hash_table[index]:
                if item[0] == key: # Anahtar bulunursa
                    return item[1] # Karşılık gelen değer döndürülür
            return None # Anahtar bulunamazsa None döndürülür

# Örnek Kullanım:
hash_table = HashTable(10) # 10 boyutunda bir hash tablosu oluşturulur
hash_table.put(10, "Ali") # Anahtar-değer çifti eklenir

```

```
hash_table.put(20, "Veli")
hash_table.put(30, "Ayşe")

print(hash_table.get(20)) # Çıktı: Veli
print(hash_table.get(40)) # Çıktı: None
```

Veli

None

Bu örnekte, HashTable adlı bir sınıf tanımlanmıştır. Bu sınıf, hash tablosunu ve hash fonksiyonunu içerir. put metodu, anahtar-değer çiftini hash tablosuna eklerken, get metodu anahtarın değerini döndürür. Hash tablosu boyutu ve hash fonksiyonu, hash tablosunun başarımılığı üzerinde önemli etkilere sahiptir.

Jump Search

Jump Search, sıralı bir dizide belirli bir öğeyi aramak için kullanılan bir arama algoritmasıdır. Bu algoritma, önce belirli aralıklarla diziyi atlayarak arama yapar, daha sonra aradığımız öğeyle karşılaşıp kadar geriye doğru lineer arama yapar.

Jump Search algoritmasının adımları şu şekildedir:

1. Dizi boyutunun kökü alınır ve bu değer, bir adımın boyutu olarak belirlenir.
2. Dizin başlangıcından başlayarak adımlar arasında gezilir. Her adımda, hedef değer kontrol edilir.
3. Eğer hedef değer bulunursa veya geçilen adımın değeri hedef değerden büyükse, geriye doğru lineer arama yapılır.
4. Hedef değer bulunana kadar bu işlem tekrarlanır. Jump Search algoritması, dizinin büyüklüğüne göre $O(\sqrt{n})$ zaman karmaşıklığına sahiptir. Lineer aramanın zaman karmaşıklığı, hedef öğe ile son adım arasındaki farka bağlıdır.

Python Kodu: Şimdi, bu açıklamalar ışığında, Python'da Jump Search algoritmasının kodunu açıklama satırlarıyla birlikte görelim:

```
In [3]: ▶ import math

def jump_search(arr, target):
    """
    Belirli bir öğeyi sıralı bir dizide aramak için Jump Search algoritması.

    Parametreler:
    arr (list): Arama yapılacak sıralı dizi.
    target: Aranılan hedef değer.

    Dönüş:
    int: Hedef değer indexi, bulunamazsa -1.
    """
    n = len(arr) # Dizi boyutu
    step = int(math.sqrt(n)) # Adım boyutu, dizinin kökü alınarak belirlenir
    prev = 0
    while arr[min(step, n) - 1] < target: # Hedef değerden küçük olan adımlar
        prev = step
        step += int(math.sqrt(n)) # Adım boyutu artırılır
        if prev >= n: # Geçilen adım dizinin sınırından büyükse
            return -1 # Hedef değer bulunamaz
    while arr[prev] < target: # Hedef değerden küçük adımlar geriye doğru
        prev += 1
        if prev == min(step, n): # Geçilen adım hedef değere ulaştıysa
            return -1 # Hedef değer bulunamaz
    if arr[prev] == target: # Hedef değer bulunduysa
        return prev # Hedef değer indexi döndürülür
    return -1 # Hedef değer bulunamaz

# Örnek kullanım:
liste = [1, 3, 5, 7, 9, 11, 13, 15, 17]
hedef = 13
sonuc = jump_search(liste, hedef)
if sonuc != -1:
    print(f"{hedef} hedefi, listede {sonuc}. indekste bulundu.")
else:
    print(f"{hedef} hedefi, listede bulunamadı.")
```

13 hedefi, listede 6. indekste bulundu.

In []: ▶